

0.23.2. Sans « x = » : la console range la position — doublé, deux positions

Les deux, sans « = » : la console range x pour le A et xb pour le B.

1. Ce qu'il faut comprendre

C'est le 0.23 et le 0.23.1 réunis : les deux lettres, sans aucun `=`.

La console range chaque position dans SA variable : celle qu'on donne en premier à `deplace_x`, `x` pour le A, `xb` pour le B.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <deplace_x>` → [0.17.3. La fonction deplace_x\(\) — avancer sur une ligne](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1147 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      66 o ← la console, à l'allumage - la mise en route de la console
deplace_x()    158 o ← main() - une fonction fournie par le langage
main()         68 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases, EcrireNombre - attend de pouvoir écrire une case
AdresseCase    10 o ← poser(), effacer() - trouve l'adresse d'une case de l'écran
EffacerCases   12 o ← effacer() - efface des cases
EcrireNombre   33 o ← nombre() - écrit un nombre en chiffres
CopierTuiles   18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         704 o ← CopierTuiles - la police entière
```

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
poser()      x 2    26 o sur place, dans deplace_x() + AdresseCase (10 o)
nombre()     x 2    20 o sur place, dans main() + EcrireNombre (33 o)
effacer()    x 1    15 o sur place, dans deplace_x() + AdresseCase (10 o), EffacerCases (12 o)
image()      x 2     6 o sur place, dans deplace_x(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
deplace_x() x 2    37 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <deplace_x> // fait avancer une tuile sur sa ligne
#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A
#include <nombre>    // écrit un nombre en chiffres
```

2. UNE VARIABLE — PAS

```
uint8_t pas = 5;
```

3. UNE VARIABLE — X

```
uint8_t x = 0; // le A
```

4. UNE VARIABLE — XB

```
uint8_t xb = 0; // le B
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    deplace_x(x, 0, ALPHABET[0], pas); // x vaut 5
    deplace_x(xb, 8, ALPHABET[1], pas); // xb vaut 5
    nombre(0, 2, x);
    nombre(0, 10, xb);

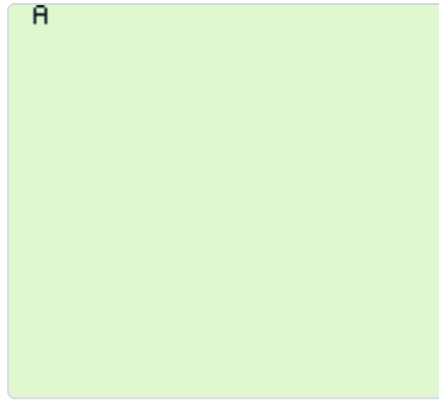
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

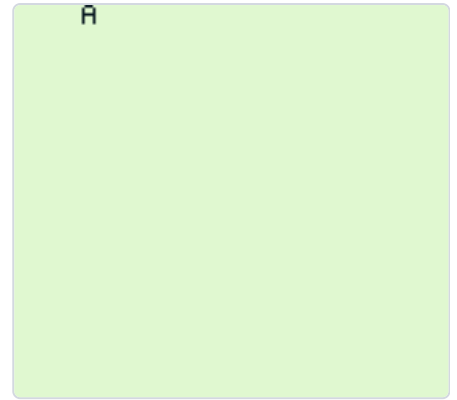
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



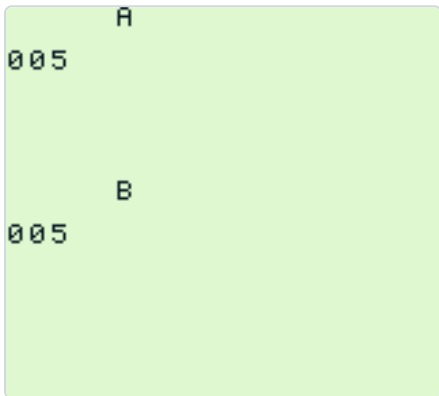
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Le A puis le B avancent de 5 cases ; sous chacun, 005.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A finit en (5, 0) (5,0)

✓ le B finit en (5, 8) (5,8)

✓ 005 et 005

Ensuite : **0.24. Les deux axes à la suite** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++